Name : Gaurav Hinduja
Div : D15C
Batch : C
Roll No : 50

MLDL Experiment 04

Aim: Implement K-Nearest Neighbors (KNN) and evaluate model performance

1. Dataset Source:

- **Dataset Name:** Breast Cancer Wisconsin Dataset
- **Source:** https://www.kaggle.com/datasets/uciml/iris
- **Original Source:** UCI Machine Learning Repository (Dr. William H. Wolberg)

2. Dataset Description:

The Breast Cancer Wisconsin dataset is used for **binary classification** of breast tumors as **benign** or **malignant**. The dataset contains features computed from digitized images of a fine needle aspirate (FNA) of a breast mass. These features describe characteristics of the cell nuclei present in the image.

Dataset Size

- **569 rows × 33 columns**

Target Variable

- **diagnosis**

Class Labels

- **M = Malignant**
- **B = Benign**

Preprocessing Note

Before training the model:

- The `id` column was removed because it does not contribute to prediction.
- The `Unnamed: 32` column was removed because it contains no useful information.
- The target variable `diagnosis` was converted into binary form:
  - **Malignant = 1**
  - **Benign = 0**

Features:-

The dataset contains numeric features such as:

1. radius_mean
2. texture_mean
3. perimeter_mean
4. area_mean
5. smoothness_mean
6. compactness_mean
7. concavity_mean

8. concave points_mean

9. symmetry_mean

10. fractal_dimension_mean

It also includes standard error and worst-value versions of these measurements.

These features are useful in identifying whether a tumor is benign or malignant.

3. Mathematical Formulation of the Algorithm:

KNN is a non-parametric, distance-based algorithm.

A. Distance Metric (Euclidean Distance):

$$d(x, x_i) = \sqrt{\sum_{j=1}^{n} (x_j - x_{ij})^2}$$

Where:

- n = number of features (4)
- x = new observation
- x_i = training sample

B. Classification Rule:

1. Compute distance between new point and all training samples

2. Select K nearest neighbors

3. Assign class using majority voting

4. Algorithm Limitations:

- Computationally expensive for large datasets
- Sensitive to feature scaling
- Performance depends on optimal choice of K

5. Methodology / Workflow:

1. Upload dataset
2. Drop Id column
3. Separate features and target
4. Apply StandardScaler
5. Split dataset (80% train, 20% test)
6. Train KNN model (K=5)
7. Evaluate using accuracy and confusion matrix
8. Tune K using Elbow Method

6. Performance Analysis:

Phase 1: Baseline KNN Model

A baseline KNN classifier was trained using:

- **n_neighbors = 5**

The model was evaluated using:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Observation:

The Breast Cancer dataset is highly suitable for KNN because it contains multiple numeric features that describe tumor characteristics. Since KNN is distance-based, **feature scaling is mandatory** to ensure that all features contribute fairly to the distance calculation.

Phase 2: Error Rate Analysis

To determine a suitable value of K, the error rate was calculated for K values from **1 to 30**.

Observation:

The **Error Rate vs K Value** graph helps identify the range of K values where the classification error is minimum. This supports selecting an optimal K value for improved prediction performance.

Phase 3: Hyperparameter Tuning

Hyperparameter tuning was performed using **GridSearchCV** to identify the best combination of:

- `n_neighbors`
- `weights`
- `metric`

Overall Observation:

The tuned KNN model performed better than the baseline model because the optimal choice of hyperparameters improved classification accuracy and model generalization.

7. Hyperparameter Tuning:

The main hyperparameter in KNN is the number of neighbors (**K**). Selecting an appropriate value of K is crucial for achieving optimal performance.

- Small K values (e.g., K = 1) may lead to **overfitting**, making the model sensitive to noise.
- Large K values may lead to **underfitting**, producing overly smooth decision boundaries.

To determine the optimal K, the **Elbow Method** was applied by computing the error rate for K values ranging from 1 to 20 and plotting the error curve.

- The value of K corresponding to the **minimum error rate** was selected as the optimal K.
- After tuning, the model achieved stable and high accuracy, confirming improved generalization performance.

Hyperparameter tuning helped in selecting a balanced K value that minimizes classification error and enhances the robustness of the KNN model.

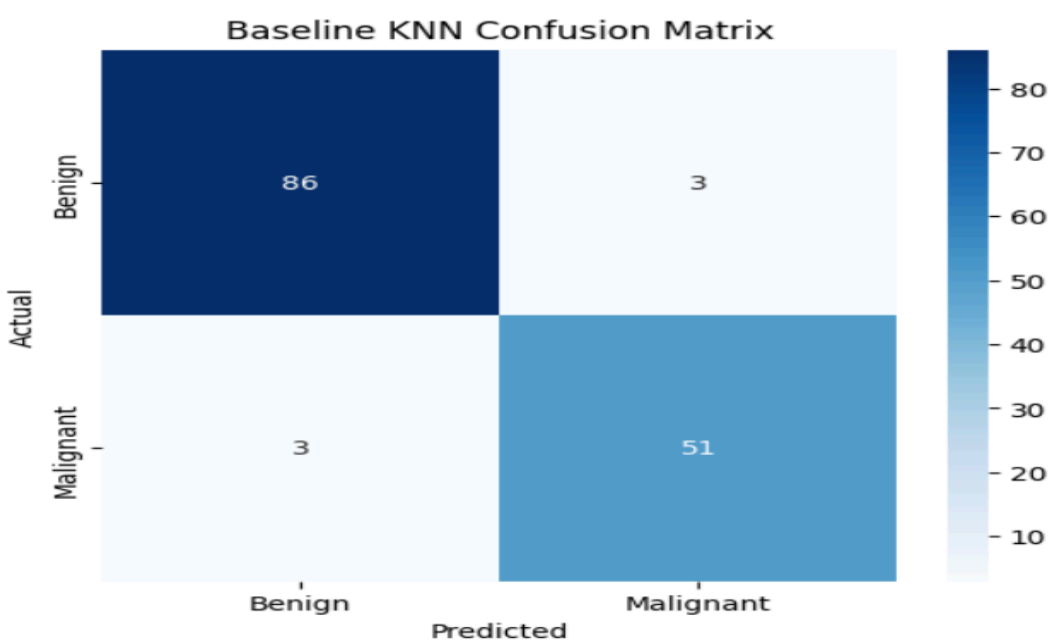
Code And Output:

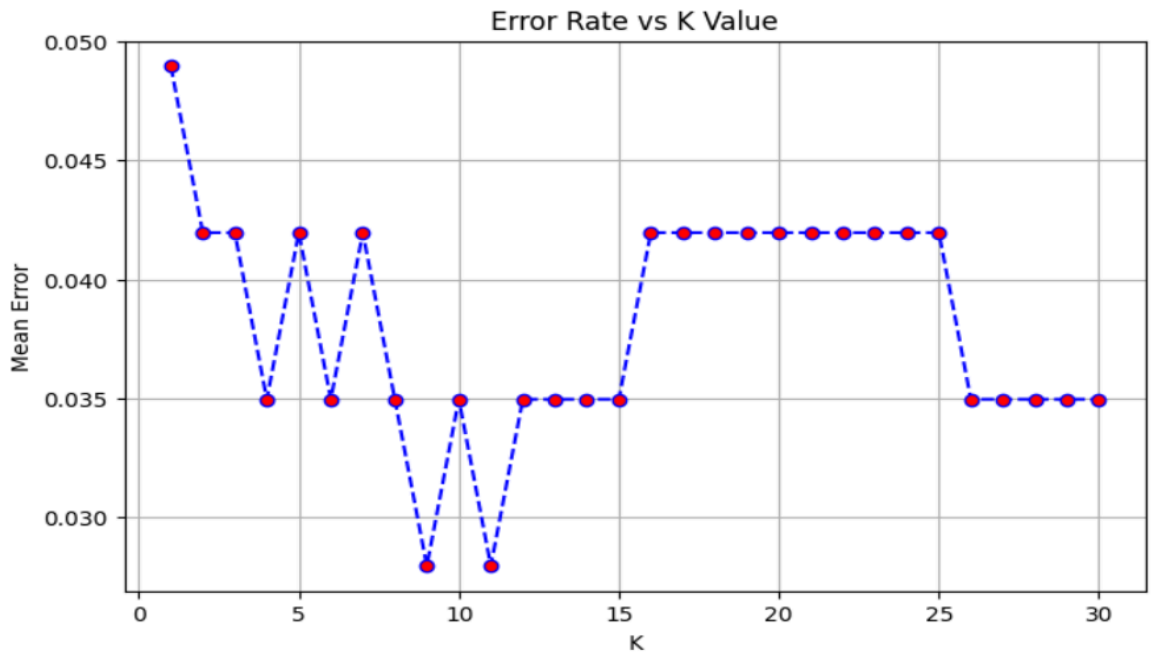
```
import pandas as pd

import numpy as np

import matplotlib.pyplot as
plt

import seaborn as sns


from google.colab import files

from sklearn.model_selection
import train_test_split

from sklearn.neighbors import
KNeighborsClassifier

from sklearn.preprocessing
import StandardScaler

from sklearn.metrics import
accuracy_score,
classification_report,
confusion_matrix


uploaded = files.upload()


filename =
list(uploaded.keys())[0]
```

```
df = pd.read_csv(filename)


if 'Id' in df.columns:

 df = df.drop('Id', axis=1)


X = df.drop('Species', axis=1)

y = df['Species']


scaler = StandardScaler()

X_scaled =
scaler.fit_transform(X)


X_train, X_test, y_train,
y_test = train_test_split(

 X_scaled, y, test_size=0.2,
random_state=42

)
knn =
KNeighborsClassifier(n_neighb
ors=5)

knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)
```

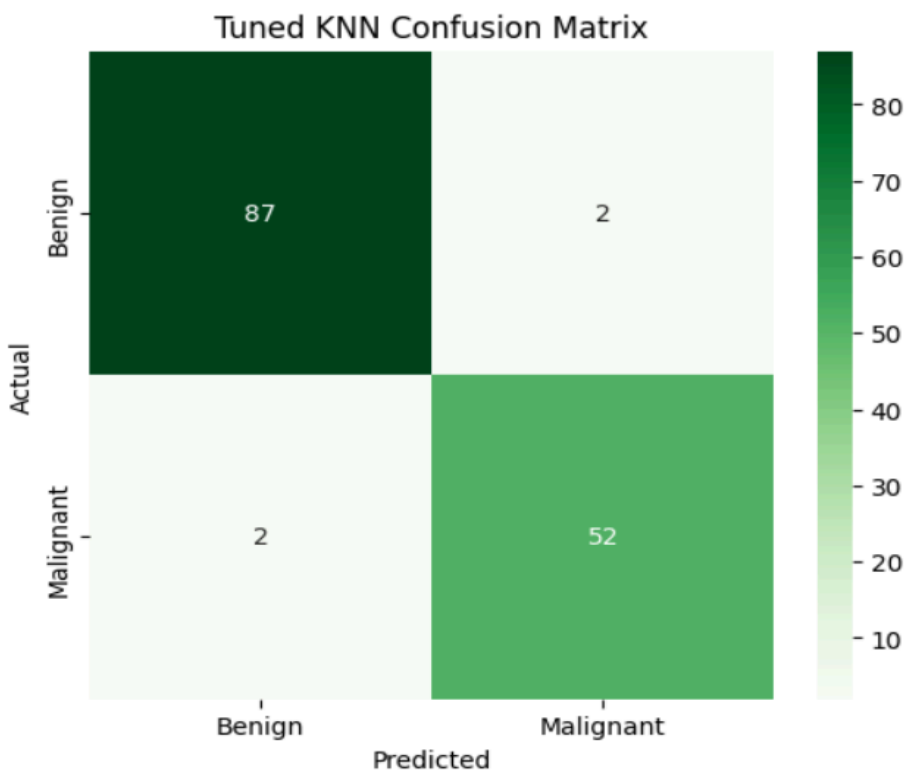
```
print("Baseline KNN (K=5)
Accuracy:",
accuracy_score(y_test,
y_pred))

print("\nClassification
Report:\n")

print(classification_report(y
_test, y_pred))


plt.figure(figsize=(6,5))

sns.heatmap(confusion_matrix(
y_test, y_pred), annot=True,
fmt='d', cmap='Purples')

plt.title("Confusion Matrix
 (K=5)")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
  error_rate = []


  for i in range(1, 20):

   knn_i =
KNeighborsClassifier(n_neighb
ors=i)

 knn_i.fit(X_train, y_train)

 pred_i =
knn_i.predict(X_test)


error_rate.append(np.mean(pre
d_i != y_test))


plt.figure(figsize=(10,6))

plt.plot(range(1,20),
error_rate,
linestyle='dashed',
marker='o')

plt.title("Error Rate vs K
Value")

plt.xlabel("K")

plt.ylabel("Error Rate")

plt.show()

optimal_k =
error_rate.index(min(error_ra
te)) + 1
  print("Optimal K:", optimal_k)
```

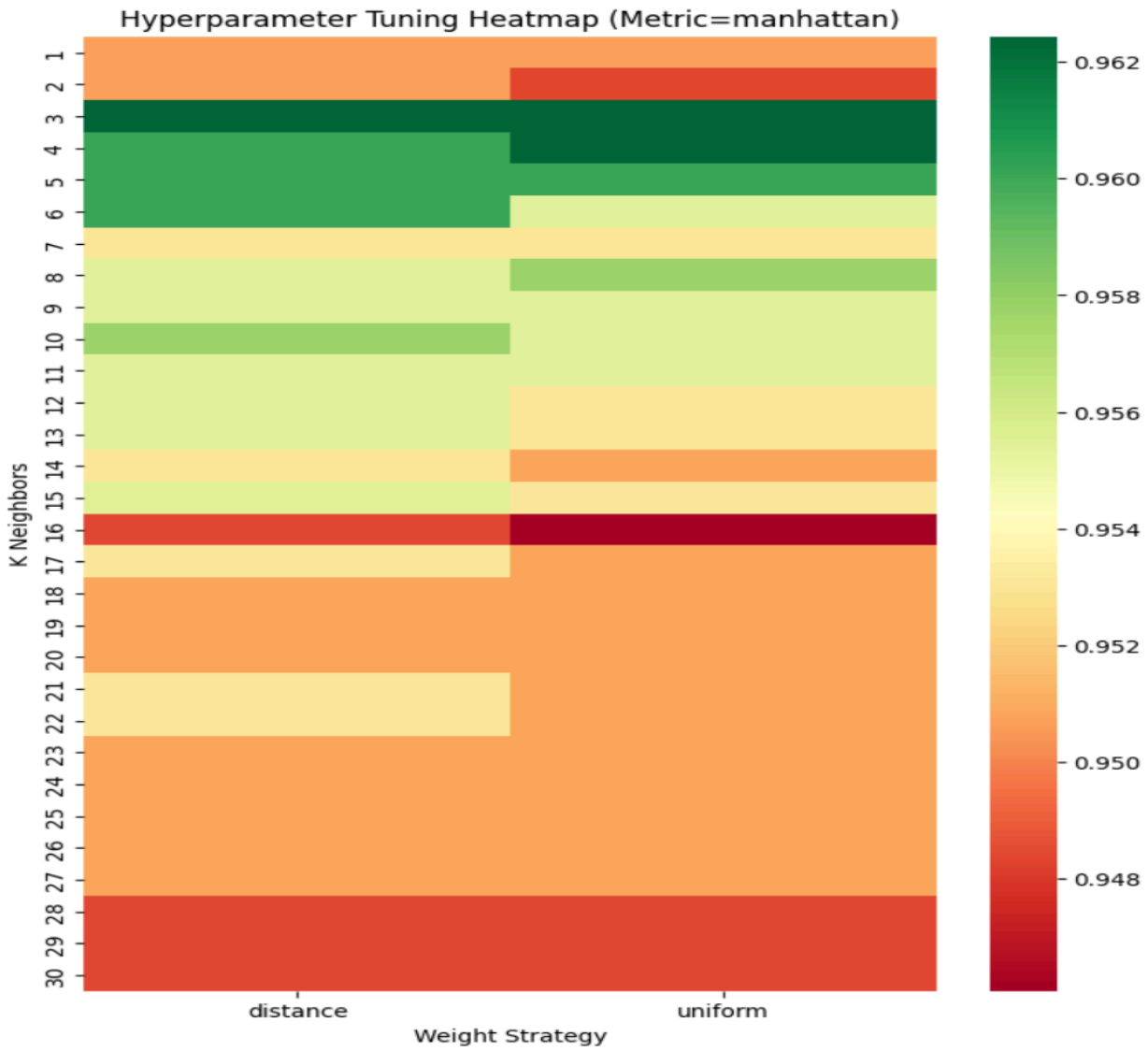
```
===== Dataset Preview =====
         id diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0    842302         M        17.99         10.38          122.80     1001.0
1    842517         M        20.57         17.77          132.90     1326.0
2  84300903         M        19.69         21.25          130.00     1203.0
3  84348301         M        11.42         20.38           77.58      386.1
4  84358402         M        20.29         14.34          135.10     1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
0          0.11840           0.27760          0.3001              0.14710
1          0.08474           0.07864          0.0869              0.07017
2          0.10960           0.15990          0.1974              0.12790
3          0.14250           0.28390          0.2414              0.10520
4          0.10030           0.13280          0.1980              0.10430

   ...  texture_worst  perimeter_worst  area_worst  smoothness_worst  \
0  ...          17.33           184.60      2019.0            0.1622
1  ...          23.41           158.80      1956.0            0.1238
2  ...          25.53           152.50      1709.0            0.1444
3  ...          26.50            98.87       567.7            0.2098
4  ...          16.67           152.20      1575.0            0.1374

   compactness_worst  concavity_worst  concave points_worst  symmetry_worst  \
0             0.6656           0.7119                0.2654          0.4601
1             0.1866           0.2416                0.1860          0.2750
2             0.4245           0.4504                0.2430          0.3613
3             0.8663           0.6869                0.2575          0.6638
4             0.2050           0.4000                0.1625          0.2364

   fractal_dimension_worst  Unnamed: 32
0                  0.11890          NaN
1                  0.08902          NaN
2                  0.08758          NaN
3                  0.17300          NaN
4                  0.07678          NaN

[5 rows x 33 columns]


--- BASELINE MATHEMATICAL RESULTS (k=5) ---
Accuracy : 95.80%
Precision: 94.44%
Recall   : 94.44%
F1 Score : 94.44%
```






```
--- TUNED MATHEMATICAL RESULTS ---
Best Parameters: {'metric': 'manhattan', 'n_neighbors': np.int64(3), 'weights': 'uniform'}
Accuracy : 97.20%
Precision: 96.30%
Recall   : 96.30%
F1 Score : 96.30%
```





Conclusion:

In this experiment, the **K-Nearest Neighbors (KNN)** algorithm was implemented on the **Breast Cancer Wisconsin Dataset** for binary classification. The goal was to classify tumors as **benign** or **malignant** using multiple numeric features extracted from breast mass cell nuclei.

The dataset was preprocessed by removing unnecessary columns such as `id` and `Unnamed: 32`, and the target variable `diagnosis` was converted into binary form where **Malignant = 1** and **Benign = 0**. Feature scaling was performed using **StandardScaler**, which is essential for KNN because it depends on distance calculations.